

ALFIDO TECH | PYTHON DEVELOPER INTERNSHIP

# Task 4 Submission

## Flask Mini Project

A full-stack web application built with Python, Flask, and Bootstrap

---

### INTERN NAME

Ishita Gupta

### INTERNSHIP TRACK

Python Developer

### TASK

Task 4 - Flask Mini Project (Interview Favorite)

### GITHUB REPOSITORY

[https://github.com/ishitagupta221401-crypto/flask\\_mini\\_project](https://github.com/ishitagupta221401-crypto/flask_mini_project)

## 1 Project Objective

This project was built to satisfy **Task 4 - Flask Mini Project** of the Alfido Tech Python Developer Internship. The brief required a simple web application demonstrating core backend fundamentals: Flask routing and templates, form handling using both GET and POST, basic CRUD functionality, and a clean Bootstrap-based interface.

Beyond the base requirements, the application was extended into a polished, production-style product named **NoteFlow**, featuring a custom dark UI design system, a notes manager with categories and timestamps, and a complete Review and Feedback module with star ratings, backend email validation, and a unique auto-generated Reviewer ID for every visitor.

## 2 Technology Stack

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| Backend Framework | Python 3, Flask (routing, Jinja2 templating)                   |
| Frontend          | HTML5, Custom CSS3 (dark theme), Bootstrap 5, Bootstrap Icons  |
| Data Handling     | In-memory storage for notes, JSON file storage for reviews     |
| Form Handling     | GET and POST methods, server-side validation with Python regex |
| Version Control   | Git and GitHub (public repository)                             |

## 3 Features Implemented

### Core Requirements (as per task brief)

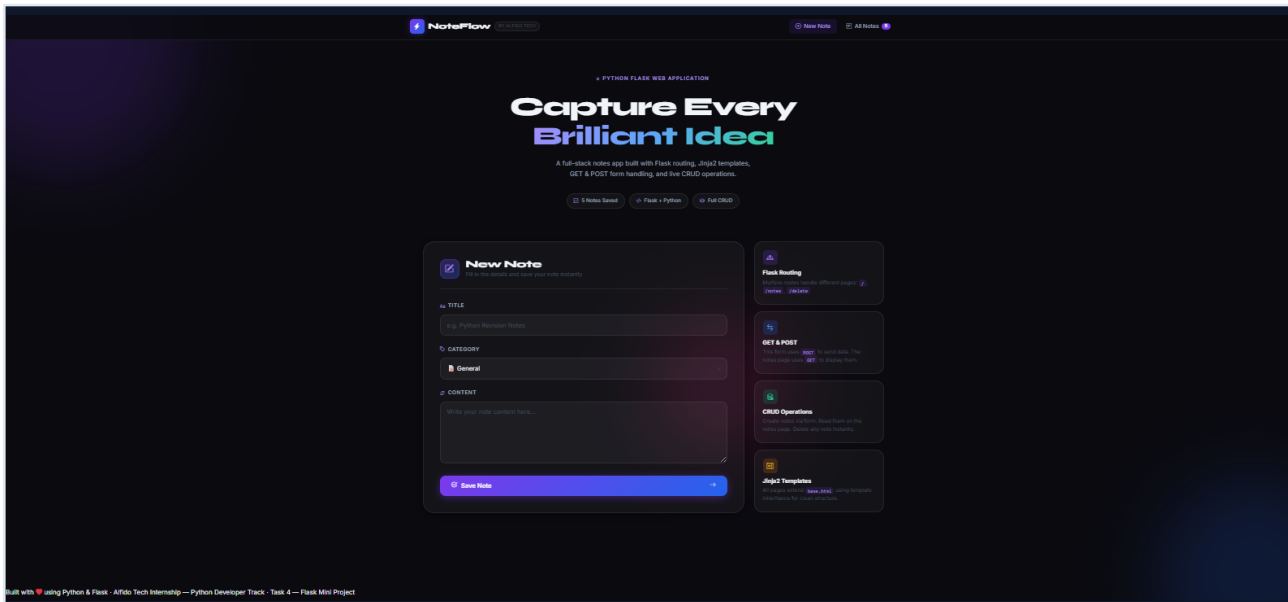
- Flask routing across multiple pages - /, /notes, /delete
- Form handling using both GET and POST HTTP methods
- Basic CRUD operations - Create, Read, and Delete notes in real time
- Clean, responsive user interface styled with Bootstrap 5

### Additional Features (beyond requirements)

- Custom dark-themed UI design system with ambient animated backgrounds
- Note categorisation (General, Study, Work, Ideas, Personal) with colour-coded badges
- Automatic date and time stamping on every saved note
- Review and Feedback system with interactive 5-star rating selector
- Backend email format validation using Python regular expressions
- Live, real-time email validation feedback on the frontend (green / red indicator)
- Unique, deterministic Reviewer ID generated for every visitor (e.g. NF-6F1C67)
- Returning-visitor detection - repeat reviewers are automatically tagged
- In-app notification system with unread badge counter and toast alerts

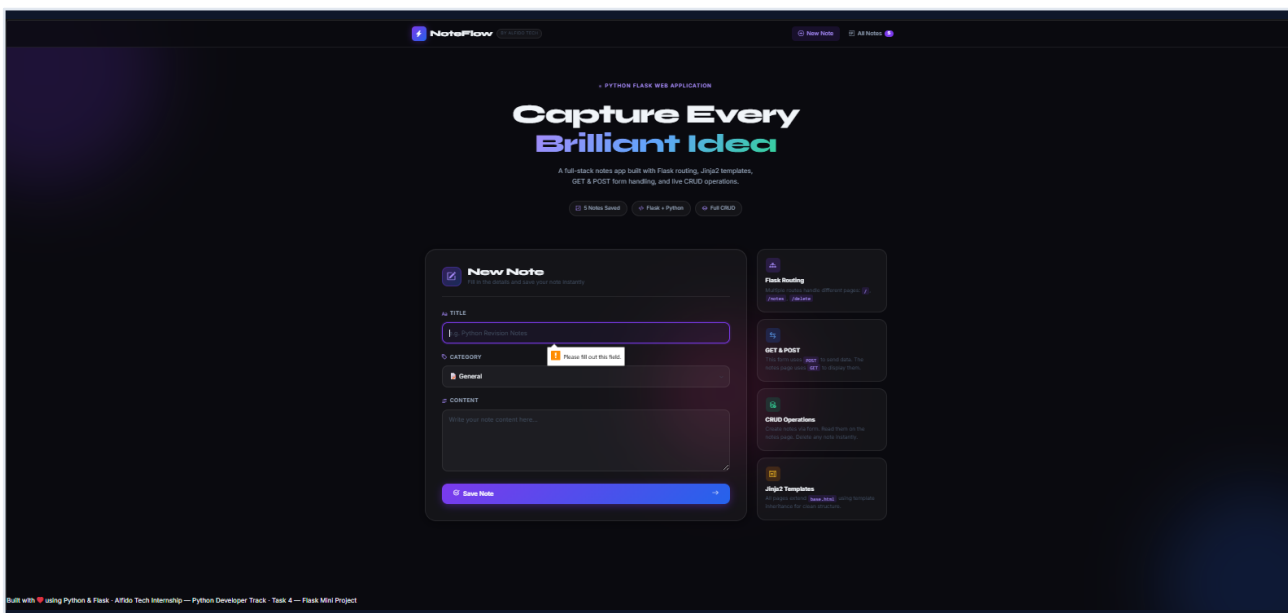
## 4 Application Screenshots

The following screenshots demonstrate the application running locally, covering routing, form handling, validation, and CRUD operations end to end.



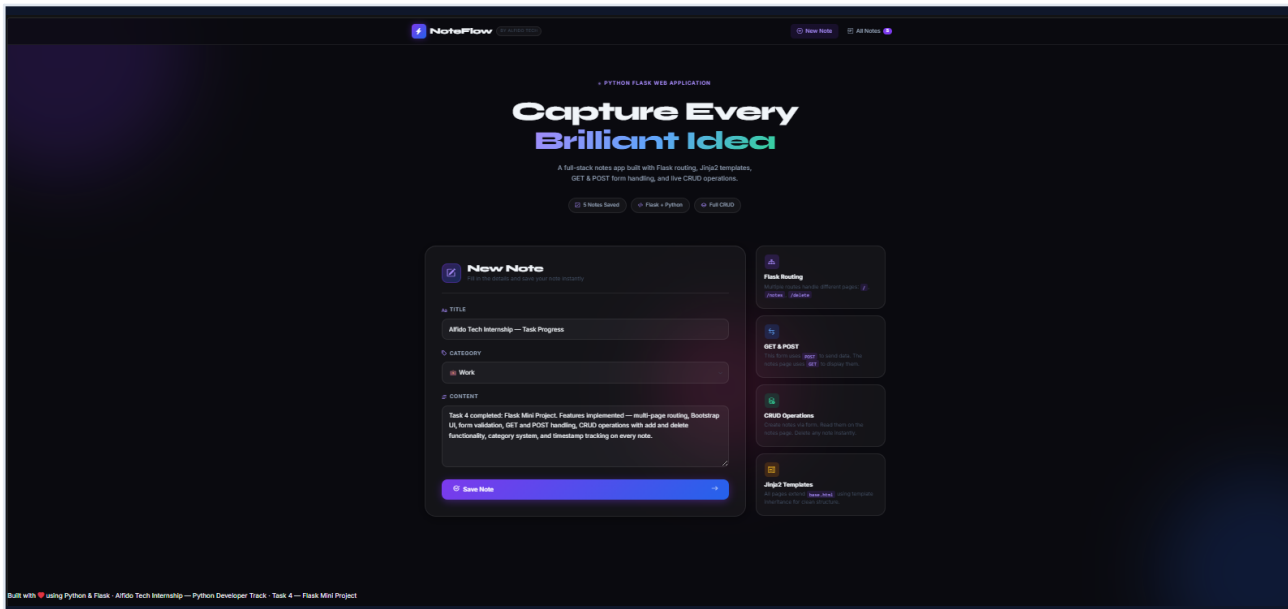
### Home Page - Note Creation Form

Landing page showing the hero section, feature highlights (Flask Routing, GET and POST, CRUD Operations, Jinja2 Templates), and the note submission form.



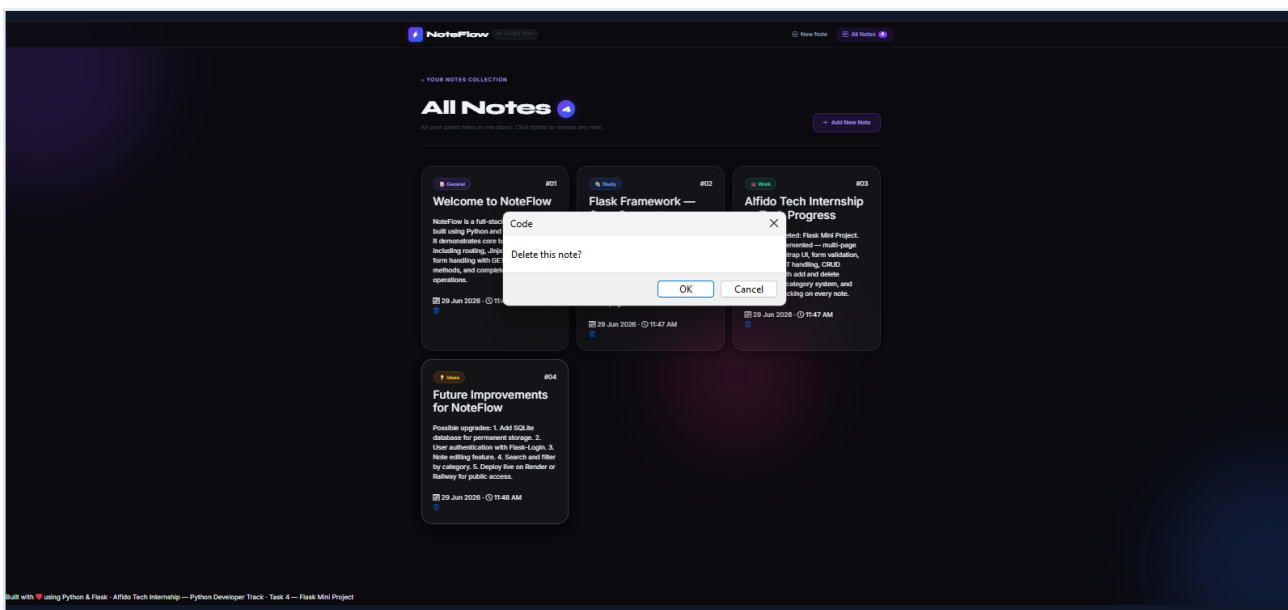
### Form Input Demonstration

The 'New Note' form being filled in with a title, category, and content, ready to be submitted via a POST request.



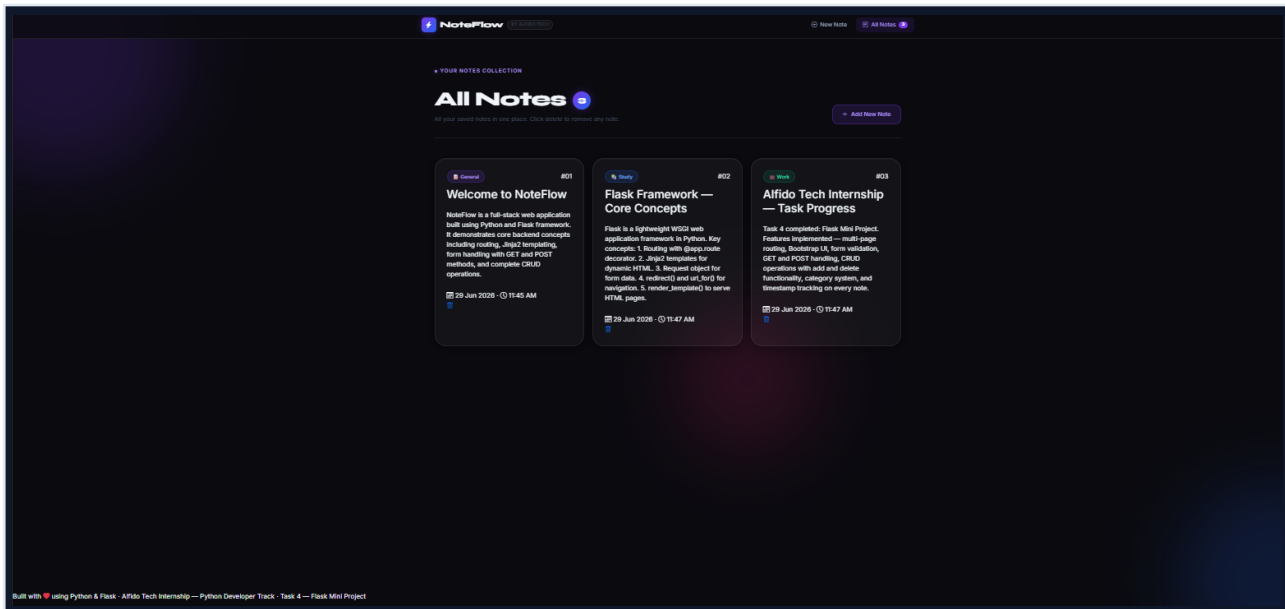
### Client-Side Form Validation

Submitting the form with an empty required field triggers built-in validation, preventing incomplete data from being sent to the server.



### All Notes - Read and CRUD View

The /notes route (GET) displaying all saved notes as styled cards, each with a category badge, timestamp, and delete action, demonstrating full CRUD.



### Delete Confirmation

A confirmation prompt appears before a note is permanently removed, preventing accidental data loss.

## 5 Key Code Snippets

### Flask Routing and CRUD Logic (app.py)

```
@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()
        content = request.form.get('content', '').strip()
        if title and content:
            notes.append({'title': title, 'content': content,
                          'date': datetime.now().strftime('%d %b %Y')})
            return redirect(url_for('view_notes'))
        return render_template('index.html')

@app.route('/notes')
def view_notes():
    return render_template('notes.html', notes=notes)

@app.route('/delete/')
def delete_note(index):
    if 0 <= index < len(notes):
        notes.pop(index)
    return redirect(url_for('view_notes'))
```

### Backend Email Validation with Regex

```
EMAIL_REGEX = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

def is_valid_email(email):
    if not email:
        return True # email is optional
    return re.match(EMAIL_REGEX, email) is not None

def generate_reviewer_id(name, email):
    seed = f"{name.strip().lower()}|{email.strip().lower()}"
    hash_digest = hashlib.sha256(seed.encode()).hexdigest()
    return f"NF-{hash_digest[:6].upper()}"
```

## 6 Repository and Submission Links

|                       |                                                                                                                                             |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| GITHUB REPOSITORY     | <a href="https://github.com/ishitagupta221401-crypto/flask_mini_project">https://github.com/ishitagupta221401-crypto/flask_mini_project</a> |
| REPOSITORY VISIBILITY | Public - viewable and clonable by anyone                                                                                                    |
| PROJECT TYPE          | Python Flask Web Application                                                                                                                |

All source code, including `app.py`, Jinja2 templates, and the custom stylesheet, is available in the GitHub repository linked above. The repository includes a complete commit history showing the project's iterative development.

## 7 Conclusion

This task strengthened practical understanding of Flask's request-response cycle, server-side rendering with Jinja2, form data handling, and designing a maintainable project structure using template inheritance. Implementing the review system additionally provided hands-on experience with data validation, persistent JSON storage, and generating unique identifiers using cryptographic hashing, concepts directly applicable to real-world backend development.